Bike Rental Prediction — R Script

Course-end Project 2 | Random Forest Regression

Code reference PDF. For execution, use the original .R file in RStudio.

```r
# ============================================================
# Bike Rental Prediction - Course-end Project 2
# Algorithm: Random Forest Regression
# Dataset: dataset.xlsx (sheet: day) or day.csv
# ============================================================

# 1. Install packages once if needed:
# install.packages(c("readxl", "dplyr", "ggplot2", "randomForest", "caret"))

# 2. Load libraries
required_packages <- c("readxl", "dplyr", "ggplot2", "randomForest", "caret")
missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]
if (length(missing_packages) > 0) {
  stop(
    "Please install these packages first: ",
    paste(missing_packages, collapse = ", "),
    "\nRun install.packages(c(",
    paste(sprintf('"%s"', missing_packages), collapse = ", "),
    ")) and rerun the script."
  )
}

library(readxl)
library(dplyr)
library(ggplot2)
library(randomForest)
library(caret)

# 3. Load data
# Keep dataset.xlsx in the same folder as this R script.
if (file.exists("dataset.xlsx")) {
  bike <- read_excel("dataset.xlsx", sheet = "day")
} else if (file.exists("day.csv")) {
  bike <- read.csv("day.csv", stringsAsFactors = FALSE)
} else {
  stop("Dataset not found. Place dataset.xlsx or day.csv in the working directory.")
}

bike <- as.data.frame(bike)

# Basic inspection
cat("\n--- Dataset dimensions ---\n")
print(dim(bike))
cat("\n--- First six rows ---\n")
print(head(bike))
cat("\n--- Structure before conversion ---\n")
str(bike)
cat("\n--- Summary ---\n")
print(summary(bike))

# 4. Data type conversion
```

```r
bike$dteday <- as.Date(bike$dteday)

# Convert categorical attributes to factors.
# Labels retain the meanings described in the project brief.
bike$season <- factor(
  bike$season, levels = c(1, 2, 3, 4),
  labels = c("Spring", "Summer", "Fall", "Winter")
)
bike$yr <- factor(bike$yr, levels = c(0, 1), labels = c("2011", "2012"))
bike$mnth <- factor(bike$mnth, levels = 1:12, labels = month.abb)
bike$holiday <- factor(bike$holiday, levels = c(0, 1), labels = c("No", "Yes"))
bike$weekday <- factor(bike$weekday, levels = 0:6,
                       labels = c("Sunday", "Monday", "Tuesday", "Wednesday",
                                  "Thursday", "Friday", "Saturday"))
bike$workingday <- factor(bike$workingday, levels = c(0, 1),
                          labels = c("No", "Yes"))
bike$weathersit <- factor(
  bike$weathersit, levels = 1:4,
  labels = c("Clear_or_partly_cloudy", "Mist_or_cloudy",
             "Light_rain_or_snow", "Heavy_rain_or_ice")
)

cat("\n--- Structure after conversion ---\n")
str(bike)

# 5. Missing value analysis
cat("\n--- Missing values by column ---\n")
print(colSums(is.na(bike)))
cat("\nTotal missing values:", sum(is.na(bike)), "\n")
cat("\n--- Missing value percentage by column ---\n")
print(round(colMeans(is.na(bike)) * 100, 2))

# 6. Monthly rental distribution (average daily rentals)
monthly_data <- bike %>%
  group_by(mnth) %>%
  summarise(avg_rentals = mean(cnt, na.rm = TRUE), .groups = "drop")

p_month <- ggplot(monthly_data, aes(x = mnth, y = avg_rentals)) +
  geom_col(fill = "steelblue") +
  labs(title = "Average Daily Bike Rentals by Month",
       x = "Month", y = "Average daily rentals") +
  theme_minimal()
print(p_month)

# 7. Yearly rental distribution (average daily rentals)
yearly_data <- bike %>%
  group_by(yr) %>%
  summarise(avg_rentals = mean(cnt, na.rm = TRUE), .groups = "drop")

p_year <- ggplot(yearly_data, aes(x = yr, y = avg_rentals)) +
  geom_col(fill = "darkseagreen4") +
  labs(title = "Average Daily Bike Rentals by Year",
       x = "Year", y = "Average daily rentals") +
  theme_minimal()
print(p_year)

# 8. Outlier analysis using boxplots
p_cnt <- ggplot(bike, aes(y = cnt)) +
  geom_boxplot(fill = "orange") +
  labs(title = "Boxplot of Total Daily Bike Rentals",
       y = "Total rentals (cnt)") +
  theme_minimal()
```

```r
print(p_cnt)

# Additional boxplots for normalized weather variables
boxplot(bike$temp, main = "Temperature", ylab = "Normalized value")
boxplot(bike$atemp, main = "Feeling Temperature", ylab = "Normalized value")
boxplot(bike$hum, main = "Humidity", ylab = "Normalized value")
boxplot(bike$windspeed, main = "Windspeed", ylab = "Normalized value")

# 9. Prepare data for Random Forest regression
# Exclude 'instant' (record index) and 'dteday' (raw date) from the model.
# Exclude 'casual' and 'registered' because their sum equals 'cnt';
# using them would cause target leakage.
model_data <- bike %>%
  select(-instant, -dteday, -casual, -registered)

# Ensure target is numeric
model_data$cnt <- as.numeric(model_data$cnt)

# Stop clearly if missing values remain in model columns
if (anyNA(model_data)) {
  stop("Missing values remain in the modeling data. Resolve them before modeling.")
}

set.seed(123)
train_index <- createDataPartition(model_data$cnt, p = 0.80, list = FALSE)
train_data <- model_data[train_index, , drop = FALSE]
test_data <- model_data[-train_index, , drop = FALSE]

cat("\nTraining rows:", nrow(train_data), "\n")
cat("Testing rows:", nrow(test_data), "\n")

# 10. Train Random Forest regression model
set.seed(123)
rf_model <- randomForest(
  cnt ~ .,
  data = train_data,
  ntree = 500,
  importance = TRUE
)

cat("\n--- Random Forest model ---\n")
print(rf_model)

# 11. Predict and evaluate model
predictions <- predict(rf_model, newdata = test_data)
actual <- test_data$cnt

rmse <- sqrt(mean((actual - predictions)^2))
mae <- mean(abs(actual - predictions))
r_squared <- 1 - sum((actual - predictions)^2) /
  sum((actual - mean(actual))^2)

cat("\n--- Test-set performance ---\n")
cat("RMSE:", round(rmse, 2), "\n")
cat("MAE:", round(mae, 2), "\n")
cat("R-squared:", round(r_squared, 4), "\n")

# 12. Actual vs predicted plot
results <- data.frame(
  Actual = actual,
  Predicted = as.numeric(predictions)
)
```

```r

p_prediction <- ggplot(results, aes(x = Actual, y = Predicted)) +
  geom_point(alpha = 0.65, color = "steelblue") +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed", color = "red") +
  labs(title = "Actual vs Predicted Bike Rentals",
       x = "Actual rental count", y = "Predicted rental count") +
  theme_minimal()
print(p_prediction)

# 13. Feature importance
cat("\n--- Variable importance ---\n")
print(importance(rf_model))
varImpPlot(rf_model, main = "Random Forest Feature Importance")

# 14. Save key outputs
write.csv(monthly_data, "monthly_rental_summary.csv", row.names = FALSE)
write.csv(yearly_data, "yearly_rental_summary.csv", row.names = FALSE)
write.csv(results, "bike_rental_predictions.csv", row.names = FALSE)

metrics <- data.frame(
  Metric = c("RMSE", "MAE", "R_squared"),
  Value = c(rmse, mae, r_squared)
)
write.csv(metrics, "model_performance_metrics.csv", row.names = FALSE)

cat("\nAnalysis complete. Summary CSV files were saved to the working directory.\n")

```